Agentic AI: Foundations

From Research Lineage to Production Patterns

Blocks 1 & 3 of 4 – KDD 2026 Tutorial, Jeju Island

Enrico Santus

Principal Technical Strategist
AI Strategy & Research Team
Office of the CTO @ Bloomberg

Based on joint work with Grace Hui Yang (Georgetown), Pranav N. Venkit (Salesforce), Hooman Sedghamiz (Bayer), Enrico Santus (Bloomberg), Victor Dibia (Microsoft Research), and Ioana Baldini (Bloomberg) – presented today by Enrico Santus, Grace Hui Yang, and Hooman Sedghamiz.

KDD 2026

Speaker Background

Enrico Santus

Principal Technical Strategist, CTO Office, **Bloomberg**
Adjunct Professor, **New York University**

- Academic affiliations and invited roles include **NYU**, **MIT**, **HK PolyU**, **SUTD**, **UniPi**, **Harvard**, **NAIST**, **King's College London**, and **University of Stuttgart**.
- Industry and research experience includes work with **Bloomberg**, **Microsoft** (**Lionbridge**), and **Bayer**.
- Research and applied expertise in **LLMs, agentic AI, NLP, AI evaluation**.

Enrico Santus

Thesis

As engineers and researchers, we should be fascinated by automation. But we should be even more fascinated by **control**.

Agentic AI is not most interesting where it maximizes autonomy, but where autonomy is carefully bounded: **scoped** by task, **constrained** by tools and permissions, **instrumented** through traces, **evaluated** over trajectories, and **governed** by explicit failure-handling policies.

Guiding question

Not simply: *How much can the system do by itself?*

But: *Under what constraints can the system act reliably?*

Learning objectives

By the end of this session, you should be able to:

1. **Define** agentic AI as LLMs embedded in loops with goals, tools, memory, feedback, and controls.
2. **Distinguish** agents from prompting, RAG, workflows, copilots, and deterministic automation.
3. **Trace** the lineage from reasoning traces to search, tool use, retrieval control, memory, and long-horizon agent prototypes.
4. **Recognize** core design patterns: planner–executor–verifier, routing, reflection, human-in-the-loop, sandboxing, fallback, and bounded tool use.
5. **Decide** when agentic control is justified by uncertainty, adaptation, dynamic tool choice, recovery, or decomposition.
6. **Explain** why production agents require safety, security, cost control, observability, and trajectory-level evaluation.
7. **Connect** this foundation to Grace Hui Yang's session on retrieval pipelines and evaluation beyond benchmarks (coming up next), to my own part on Agents in Finance, and to Hooman Sedghamiz's closing session on deploying agentic AI in life sciences and enterprise settings.

Sources: Goals mirror [Anthropic, 2024, Yang et al., 2026]; evaluation caution from [Kapoor et al., 2024, Yehudai et al., 2026].

Roadmap

1. Block 1, Part 1: Agentic Systems – History and Definitions
2. Block 1, Part 2: Research Lineage: Reusable Primitives
3. Block 1, Part 3: Agent Architectures & Design Patterns
4. Block 3: Agents in Finance

What is an agent? A pre-LLM concept

Classical definition

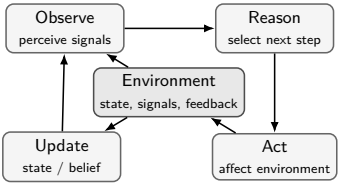
An **agent** is a system that **perceives** an environment and **acts** in that environment to pursue goals.

Core ingredients

- environment
- observations
- actions
- goals or utility
- feedback

The term *agent* predates LLMs by decades.

Canonical loop



Historical point

LLM agents did not invent agency; they changed the substrate through which agency is implemented.

Sources: [Russell and Norvig, 2020, Messing, 2002].

What changed with large language models?

The interface changed

LLMs made **language** a flexible interface for goals, task decomposition, tool use, feedback, and memory.

Classical agent systems

- explicit state representations →
- explicit planners →
- predefined action spaces →
- hand-engineered policies →
- domain-specific interfaces →

LLM-based agent systems

- natural-language goals
- language-based task decomposition
- open-world tool calls through schemas/APIs
- feedback interpreted as text
- flexible cross-domain control

Modern shift

The LLM becomes a **control layer** between the user, the task, tools, retrieval, memory, and external environments.

Sources: [Vaswani et al., 2017, Wei et al., 2022, Schick et al., 2023].

Why LLM agents need architecture

Core idea

LLMs are flexible controllers, but they are not reliable systems by themselves.

LLM limitations

- incomplete or stale knowledge →
- unreliable computation →
- no native persistent state →
- hallucination under uncertainty →
- brittle long-horizon behavior →
- limited context window →

Architectural responses

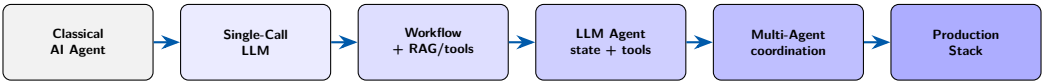
- retrieval and grounding
- tools and code execution
- memory and state management
- verification and critique
- planning with checkpoints
- context management

Design principle

Agent components are not decorative. Each one should control a specific failure mode.

Sources: [Wei et al., 2022, Yao et al., 2022, Schick et al., 2023, Zhao et al., 2025].

The architecture spectrum: Not a binary



Workflow

LLMs are orchestrated through mostly **predefined code paths**.

Agent

The system can **dynamically direct** parts of its own process and tool use.

Engineering question

Not: *Is this really an agent?*
But: *How much autonomy is justified, observable, and controlled?*

Sources: [Russell and Norvig, 2020, Anthropic, 2024, Yang et al., 2026].

Working definition: Capabilities & Obligations

Agentic system

An LLM-based system that **repeatedly selects actions** to advance a goal, using reasoning, tools, memory, environment feedback, and control policies.

Capabilities

- decompose goals
- gather context
- choose tools
- track state
- verify progress

Obligations

- bound permissions
- log trajectories
- manage memory
- recover or roll back
- escalate to humans

Agency increases both capability and risk

Autonomy spectrum: **assist** (human decides) → **delegate** (human monitors) → **autonomous** (human audits). Rising autonomy creates new system obligations.

Sources: [Zhao et al., 2025, Huang et al., 2024, Kapoor et al., 2024, Anthropic, 2024].

Why now? Three converging forces

Stronger reasoning

- Chain-of-Thought [Wei et al., 2022]
- Self-Consistency [Wang et al., 2022]
- Tree-of-Thoughts [Yao et al., 2023]
- Reflection and self-critique [Shinn et al., 2023]

Better tool use

- Toolformer [Schick et al., 2023]
- Code interpreters
- Browser automation
- MCP standard [Gullí, 2025]

Realistic environments

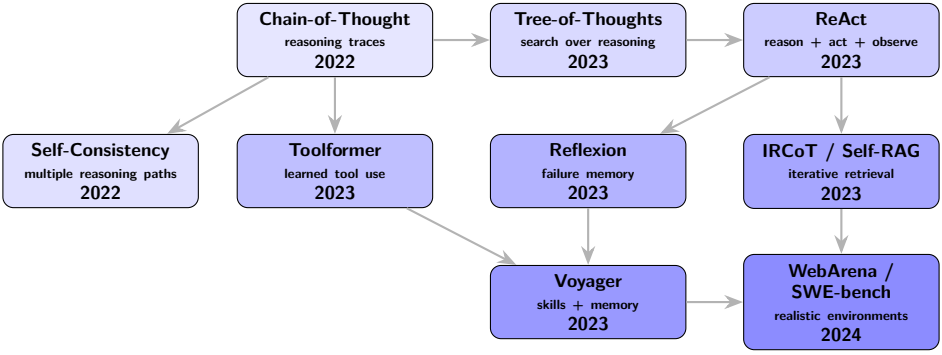
- WebArena [Zhou et al., 2024]
- SWE-bench [Jimenez et al., 2024]
- WorkArena [Drouin et al., 2024]
- CRMArena [Huang et al., 2025]

The real unlock

Not just smarter models — but models embedded in **loops with tools, state, and feedback** operating in **environments that can be measured**.

Roadmap

Research lineage: Reusable primitives



Reading the lineage

Each work introduced a reusable **primitive**: traces, search, action, tools, reflection, retrieval control, memory, or environment-based evaluation.

Sources:

[Wei et al., 2022, Wang et al., 2022, Yao et al., 2023, Yao et al., 2022, Schick et al., 2023, Shinn et al., 2023, ?, Asai et al., 2023, Wang et al., 2024, Zhou et al., 2024, Jimenez et al., 2024].

Chain-of-Thought: Reasoning traces

Wei et al., Google Research / Brain Team, 2022

Chain-of-thought prompting adds intermediate reasoning steps to few-shot examples, so the model generates a reasoning path before the final answer.

What the paper changed

- Standard prompting: question to answer
- CoT: question to reasoning trace to answer
- Uses few-shot exemplars, not fine-tuning
- Works best at large model scale

Empirical anchor

- Benchmark: GSM8K math word problems
- Model: PaLM 540B
- CoT vs. Standard: 56.9% vs. 17.9%

Agentic primitive

- reasoning trace
- scratchpad
- inspectable intermediate state
- basis for later action loops

Why it matters for agents

Chain-of-thought is not yet agency, but it gives agentic systems a manipulable internal object: a reasoning trace.

Sources: [Wei et al., 2022].

Example: Chain-of-Thought

Task

Roger has 5 tennis balls. He buys 2 cans of tennis balls. Each can has 3 balls. How many balls does he have?

Direct answer

- Roger has 11 balls.

Problem

- The answer is correct here.
- But the reasoning path is hidden.
- We cannot inspect intermediate steps.

Chain-of-thought answer

- Roger starts with 5 balls.
- He buys 2 cans.
- Each can has 3 balls.
- 2 cans contain 6 new balls.
- 5 plus 6 is 11.

Primitive

The model produces a reasoning trace before the final answer.

Self-Consistency: Sampling reasoning paths

Wang et al., Google Research / Brain Team, 2022

Self-consistency replaces one greedy chain-of-thought with multiple sampled reasoning paths, then selects the most consistent final answer.

Problem addressed

- CoT depends on one reasoning path.
- A wrong first step can yield a fluent but incorrect answer.
- Self-consistency: reasoning as a set of candidate paths.

Method

- Sample multiple chain-of-thought paths.
- Extract the final answer from each path.
- Return the most frequent final answer.

Reported gains over CoT

- GSM8K: +17.9, SVAMP: +11.0
- AQuA: +12.2, StrategyQA: +6.4
- ARC-Challenge: +3.9

Agentic primitive

- multiple candidate trajectories
- aggregation before commitment
- ensemble, debate, and verifier patterns

Sources: [Wang et al., 2022].

Tree-of-Thoughts: Reasoning as search

Yao et al., Princeton and Google DeepMind, 2023

Tree-of-Thoughts treats reasoning as search over intermediate "thoughts," rather than a single left-to-right chain.

Problem addressed

- CoT usually follows one path.
- Early mistakes can lock in a bad trajectory.
- Some tasks require exploration, lookahead, and backtracking.

Method

- Generate several candidate thoughts.
- Evaluate progress and keep promising paths.
- BFS or DFS search, and backtrack if needed.

Concrete result

- Task: Game of 24, with GPT-4
- ToT vs. CoT: 74% vs. 4%

Agentic primitive

- reasoning as search
- candidate generation
- evaluator step
- planner-executor pattern
- recovery from bad paths

Sources: [Yao et al., 2023].

Example: Tree-of-Thoughts

Task

Use 4, 4, 6, and 8 to make 24.

Single-path reasoning

- Follow one candidate path.
- Early mistakes can lead to a dead end.

Dead end

- $8 - 6 = 2$
- $4 + 4 = 8$
- $8 \times 2 = 16$

Tree-of-Thoughts

- Generate several candidate steps.
- Evaluate candidate branches.
- Backtrack when needed.

Successful branch

- $8 + 4 = 12$
- $6 - 4 = 2$
- $12 \times 2 = 24$

Primitive

Reasoning becomes search over intermediate states.

ReAct: The reasoning–acting loop

Yao et al., Princeton and Google, 2023

ReAct interleaves reasoning traces with task-specific actions and environment observations.

Problem addressed

- CoT reasons before answering.
- CoT does not interact with the environment.
- It can hallucinate or propagate mistakes.
- ReAct: reason, act, observe, and update.

Method

- Thought: decide what is needed.
- Action: call a tool or act in the environment.
- Observation: read the result.
- Thought/Finish: update or complete.

Concrete result

- Tasks: HotpotQA and FEVER
- ReAct: Wikipedia API for evidence
- Improved interpretability & trustworthiness
- ALFWorld: +34%, WebShop: +10%

Agentic primitive

- reason-act-observe loop
- tool use with feedback
- execution trace
- recovery from observations

Sources: [Yao et al., 2022].

Example: ReAct

Task

Who is the author of the novel that introduced the character Sherlock Holmes?

Reasoning and action

- Thought: Identify the first SH novel.
- Action: Search for first SH novel.
- Observation: The novel is A Study in Scarlet.

Continue

- Thought: Author of that novel.
- Action: Search for the author.
- Observation: He is Arthur Conan Doyle.

Final response

- The author is Arthur Conan Doyle.

Why this differs from CoT

- The model does not rely only on memory.
- It acts in an external environment.
- Observations update the next step.

Primitive

Reasoning, action, and observation become one execution loop.

Toolformer: Models learn to call tools

Schick et al., Meta AI, 2023

Toolformer trains a language model to decide when to insert API calls, what arguments to pass, and how to use the returned result.

Problem addressed

- LMs are strong few-shot learners.
- Arithmetic and factual lookup remain difficult.
- Specialized tools can solve them.
- Toolformer learns when to delegate.

Method

- Start from a few tool-use demonstrations.
- Propose API calls in raw text (execute and insert results). Keep them if they improve next-token prediction.

Tools used

- calculator, QA system, calendar
- two search engines
- translation system

Agentic primitive

- tool name and arguments
- external observation
- result integrated into generation
- tool call as action

Reflexion: Learning from failure

Shinn et al., Northeastern, Princeton, 2023

Reflexion agents improve through verbal feedback: they reflect on failures and store lessons in episodic memory.

Problem addressed

- LLM agents cannot easily update weights.
- Trial-and-error feedback is still useful.
- Feedback must be reusable at runtime.

Method

- Attempt a task and receive feedback.
- Write a reflection on failure and store it in memory.
- Retry with that memory in context.

Concrete result

- HumanEval programming tasks
- GPT-4 pass@1: about 80% to 91%
- Also improved ALFWorld & HotpotQA

Agentic primitive

- runtime adaptation
- verbal failure memory
- retry loop
- improvement without weight updates

Sources: [Shinn et al., 2023].

Example: Reflexion

Task

Write a function that returns the maximum number in a list.

First attempt

- The agent writes code.
- The code fails on an empty list.
- Feedback: test failed for empty input.

Reflection

- I forgot to handle the empty-list case.
- Next time, check whether the list is empty before calling max.

Retry

- The agent includes the reflection in memory.
- It rewrites the function.
- It adds an empty-list guard.
- The new attempt passes the test.

What changed

- No model weights changed.
- The agent adapted through verbal memory.

Primitive

Failure feedback becomes a stored lesson used in the next attempt.

IRCoT / Self-RAG: Retrieval as a control loop

Trivedi et al., 2023; Asai et al., 2024

Retrieval becomes adaptive: the model decides what evidence it needs, retrieves it, and updates or critiques its answer.

Problem addressed

- Standard RAG retrieves once.
- Multi-step tasks need evolving evidence.
- Retrieved passages may be irrelevant.

IRCoT

- Reason, then retrieve.
- Continue with new evidence.

Self-RAG

- Decide when to retrieve.
- Generate with evidence.
- Critique relevance and support.

Agentic primitive

- adaptive evidence-seeking loop
- grounding self-critique

Key idea

Knowledge retrieval becomes part of the reasoning trajectory.

Sources: [?, Asai et al., 2023].

Voyager: An integrated long-horizon agent

Wang et al., NVIDIA, Caltech, UT Austin, Stanford, ASU, 2023

Voyager is an LLM-powered Minecraft agent that combines goal selection, code actions, feedback, memory, and skill reuse.

System components

- Automatic curriculum proposes goals.
- Code generation writes executable skills.
- Environment feedback reports results/errors.
- Self-verification checks goal success.
- Skill library stores reusable behaviors.

Why more than planning

- It executes code in the world.
- It fixes failures from feedback.
- It reuses skills in new settings.

Concrete result

- 3.3× more unique items, 2.3× longer travel distances, and up to 15.3× faster milestones.
- Uses GPT-4 through black-box queries.
- No parameter fine-tuning.

Agentic primitive

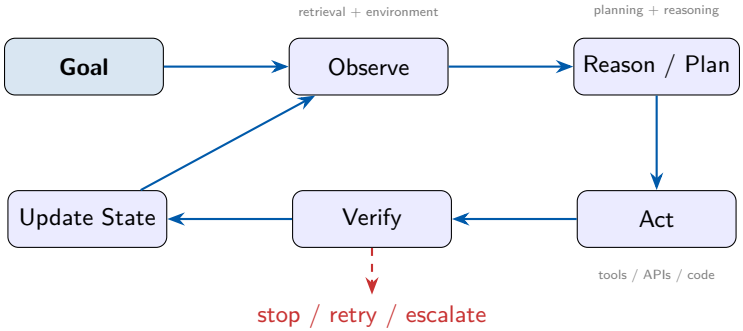
- long-horizon goal pursuit, code-as-action
- feedback loop, executable memory
- iterative refinement

Sources: [Wang et al., 2024].

Roadmap

1. Block 1, Part 1: Agentic Systems – History and Definitions
2. Block 1, Part 2: Research Lineage: Reusable Primitives
3. Block 1, Part 3: Agent Architectures & Design Patterns
4. Block 3: Agents in Finance

The canonical agent loop



Every arrow is a failure surface

Goal ambiguity, stale observations, bad plans, unsafe tools, weak verification, or corrupted memory can compound across the loop; **this is why we debug agent trajectories, not just outputs.**

Sources: [Yao et al., 2022, Shinn et al., 2023, Zhao et al., 2025].

Context windows: The agent's immediate working memory

What lives in context

- system prompt and policies
- tool schemas and permissions
- conversation / task history
- retrieved documents
- tool outputs and errors
- current plan, state, and scratchpad

Management strategies

- summarize older history
- retrieve selectively
- externalize long-term memory
- decompose into subproblems
- compact traces and tool outputs
- checkpoint and resume state

Design implication

Context determines what the agent can attend to, reason over, and act on at each step.

Bridge to production

Long-running agents are not just long prompts. They require infrastructure for state, memory, monitoring, recovery, and control.

Sources: [Zhao et al., 2025, Wei et al., 2022, Yao et al., 2022, Wang et al., 2024].

Memory: Capability and risk surface

Type	Stores	Agent examples	Primary risk
Working	task state, plan, scratchpad	CoT scratchpad; ReAct trace	overflow, contradiction
Episodic	prior attempts, failures, skills	Reflexion memory; Voyager skill library	stale or poisoned lessons
Semantic	durable facts, documents, domain knowledge	RAG knowledge base; domain memory	provenance loss, update lag

Design principle

Memory should be **inspectable**, **scoped**, **versioned**, and **revocable**.

Key point

Memory extends agency over time. It can preserve useful lessons, but also false assumptions, stale evidence, private data, or poisoned state.

Sources: [Wei et al., 2022, Yao et al., 2022, Shinn et al., 2023, Wang et al., 2024].

Planning: Turning goals into safe execution

Why planning matters in agent architecture

An agent must decide **what to do next**, **in what order**, and **when to stop**.

A language plan

- coherent & reasonable steps
- may hide missing information
- may ignore tool limits
- may fail during execution

An executable plan

- explicit actions, inputs, and outputs
- respects constraints
- checks progress after each step
- can stop, retry, or replan

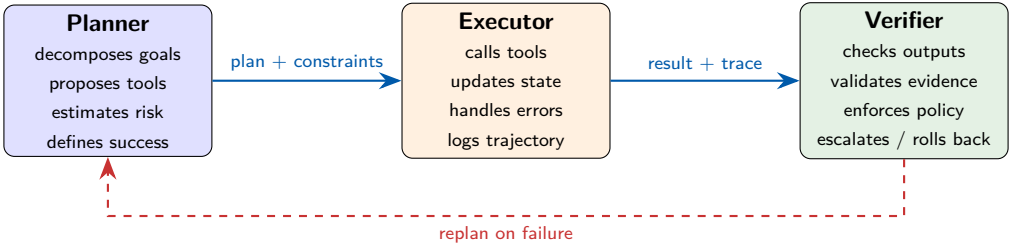
Simple example

"Book my trip." An executable plan must know the dates, budget, constraints, and when to trigger human approval.

Practitioner rule

Keep LLM planning **bounded**: use a limited horizon, explicit tools, clear budgets, and explicit stopping conditions.

Planner–Executor–Verifier



Why split roles?

Role separation creates independent checkpoints. The planner can be creative, the executor can be constrained, and the verifier can be stricter than both. This reduces self-confirmation and makes the trajectory easier to audit.

Sources: Role-based agents and orchestration: [Wu et al., 2023, Fourney et al., 2024].

Pattern catalog: Complexity should earn its place

Pattern	Use when	Watch out for	Example
Router	tasks map to specialists	brittle classification	ticket triage
Reflection	revision is cheap	self-confirmation	code review
Plan–execute	task is multi-step	plan drift, cost	research assistant
Critic / verifier	correctness is checkable	verifier blind spots	math / code checks
Human gate	actions are high-stakes	approval fatigue	trade execution
Sandbox	code, browser, or tools execute	environment mismatch	SWE-bench solver
Fallback	failure is expected	silent degradation	support workflows

Design principle

Start with the simplest architecture that works. Add agentic complexity only when the task requires adaptation, tool choice, feedback, or recovery.

Sources: Patterns adapted from [Anthropic, 2024]; related primitives: [Yao et al., 2022, Shinn et al., 2023, Wu et al., 2023].

When to use agents vs. simpler systems

Use an agent when

- observations are partial or changing
- tool choice is context-dependent
- feedback is available during execution
- failures can be detected and recovered
- user intent requires clarification or adaptation

Prefer a workflow when

- task steps are stable and known
- schema is fixed in advance
- failures are costly or irreversible
- latency and auditability dominate
- deterministic automation already works

Sober takeaway

Many successful "agent" products are constrained workflows with a few agentic decision points. The best architecture is the simplest one that solves the task under realistic conditions.

Sources: [Anthropic, 2024, Kapoor et al., 2024].

From single agents to coordinated systems

Why coordinate agents?

- **Specialization**: different roles, tools, or knowledge
- **Parallelism**: independent subtasks can run concurrently
- **Critique**: disagreement can expose errors
- **Organizational fit**: agents can mirror human workflows

Why coordination is hard

- sequential tasks may gain little from parallelism
- one agent's error can become another's premise
- shared model biases can create false consensus
- coordination can dominate task cost
- state becomes distributed and harder to debug

Design principle

Multi-agent systems are justified when the task benefits from **decomposition, specialization, parallelism, or independent critique**. More agents do not automatically mean higher robustness.

Sources: [Wu et al., 2023, Tran et al., 2025, Fourney et al., 2024, Zhu et al., 2025].

Coordinating multi-agent systems: Topologies, protocols, and fault tolerance

Four design dimensions the field is converging on

- **Architectural topology** – who talks to whom (hierarchical vs. peer-to-peer)
- **Communication protocol** – what is exchanged and how
- **Negotiation vs. cooperation** – do agents compete for resources or align on a shared objective
- **Fault tolerance** – how the system detects and survives a failing agent

Topology	Strength	Risk	Example
Hierarchical	clear delegation and escalation	bottleneck, manager error	Magentic-One
Peer-to-peer	flexible collaboration	loops, deadlocks	AutoGen chat
Blackboard	shared workspace	memory pollution	shared scratchpad
Debate / critic	error discovery	performative disagreement	AI Co-Scientist
Market / auction	task allocation	incentive design complexity	resource scheduling

Sources: [Wu et al., 2023, Tran et al., 2025, Fourney et al., 2024, Zhu et al., 2025, Gottweis et al., 2026].

Failure modes and mitigations

Failure mode	What it looks like	Mitigation
Hallucination	confident but ungrounded claims; fabricated tool results or citations	cross-checking against retrieved evidence; verification pipelines
Deadlock	agents wait on each other, or a single agent loops without progress	step/turn budgets; timeout and escalation policies
Drift	the system gradually departs from the original goal or policy	periodic goal re-grounding; planner–verifier checkpoints
Cascading errors	one agent's mistake becomes the next agent's premise	role-separated verification; human-in-the-loop fallback

Running theme

These four failure modes recur in single-agent and multi-agent systems alike. No single technique fixes all of them: reliability comes from layering cross-checks, verification, and human fallback at the points where failure is most costly.

Sources: [Yang et al., 2026, Kapoor et al., 2024, Mozannar et al., 2025, Venkit et al., 2025].

Take-home principles

1. **Autonomy is not the objective.** Appropriate, bounded autonomy is.
2. **Agents are systems, not prompts.** They require state, tools, policies, verification, and recovery.
3. **The trajectory is the unit of analysis.** Inspect plans, actions, observations, memory, and verification – not just the final answer.
4. **More agents do not automatically mean higher robustness.** Coordination must earn its cost.
5. **Failure is a first-class design concern.** Hallucination, deadlock, drift, and cascading errors are addressed by cross-checking, verification, and human-in-the-loop fallback – not hoped away.

Coming up next: Grace Hui Yang

Retrieval Pipelines & Evaluation Beyond Benchmarks

Roadmap

1. Block 1, Part 1: Agentic Systems – History and Definitions
2. Block 1, Part 2: Research Lineage: Reusable Primitives
3. Block 1, Part 3: Agent Architectures & Design Patterns
4. Block 3: Agents in Finance

Welcome back: Agents in Finance

From agent architectures to financial research

Finance is a consequential domain: a bad agent trajectory can become a wrong research conclusion, a misleading signal, or an expensive decision.

- The question is not simply: *Can LLMs predict markets?*
- The better question: *Can agents help automate parts of the financial research loop?*
- The constraint: every result must be **point-in-time valid**, reproducible, and auditable.

Thesis for this block

Agentic AI is changing finance by turning static tools into **controlled research systems**.

What financial analysts already do

Analyst research loop

1. Ask a research question
2. Gather data and evidence
3. Form a hypothesis
4. Implement the analysis
5. Backtest and diagnose
6. Revise and remember

Agentic translation

- **Planner:** decide what to investigate
- **Retriever:** find data, news, filings, reports
- **Coder / executor:** write and run analysis
- **Verifier:** check leakage, evidence, logic
- **Memory:** preserve lessons from attempts
- **Human gate:** approve consequential use

Key framing

Agentic AI does not invent financial research. It begins to **automate and connect steps analysts already perform**.

Running example: from hypothesis to alpha factor

Simple research question

Do stocks with **improving momentum** and **falling volatility** tend to outperform?

Plain-English idea

- Momentum: price has recently improved
- Volatility: price has become less unstable
- Alpha factor: a rule that gives each stock a score
- Backtest: simulate whether high-score stocks later outperform

Toy scoring rule

```
score(stock) =
    recent_return(stock)
    - recent_volatility(stock)
```

- Rank stocks by score
- Test only with information available at time t

Why agents matter

An agent can propose variants, implement them, run tests, diagnose failures, and remember what worked.

Trend 1: Automating the experimental cycle

What the agent starts to automate

- inspect a dataset
- propose experiments
- write analysis code
- build and test the evaluator
- run large experiment campaigns
- record successful and failed attempts

Research example: AlphaLab

- Builder / Critic / Tester: construct and audit evaluation
- Strategist / Worker: choose and run experiments
- Playbook: persistent experimental memory

Innovation

The agent automates the **loop around the experiment**, not just a single model call.

Finance implication

If the evaluator is flawed, the agent can optimize the flaw. Verification must be part of the research harness.

Sources: AlphaLab: [Hogan et al., 2026].

Trend 2: Expanding the search for alpha

From formulas to richer search

- Formula: `momentum - volatility`
- Factor code: executable stock-scoring logic
- Research trajectory: hypothesis → code → backtest → diagnosis → revision

Why this matters

- Analysts cannot manually test every plausible signal.
- Agents can generate, modify, combine, and evaluate candidates.

Sources: CogAlpha: [Liu et al., 2026b]; QuantaAlpha: [Han et al., 2026].

Supporting papers

- **CogAlpha:** evolves executable factor code.
- **QuantaAlpha:** evolves the full research trajectory.

Simple distinction

CogAlpha changes the **candidate**.
QuantaAlpha changes the **process that produced the candidate**.

Example: factor code vs. research trajectory

Factor code: one candidate

```
if volatility_falling(stock):
    score = momentum(stock)
else:
    score = 0
```

- Gives each stock a score
- Backtest checks whether high-score stocks outperform
- Mutation changes the rule; crossover combines rules

Trajectory: the whole attempt

1. Hypothesis: momentum works when volatility falls
2. Code: implement momentum + volatility filter
3. Backtest: weak performance
4. Diagnosis: volatility window too slow
5. Revision: shorten the volatility window

Innovation

The system can revise the weak step without discarding the whole idea.

Trend 3: Agents that read, remember, and reuse research

The analyst problem

- Good research uses prior literature and prior failures.
- Most model calls start from a nearly blank slate.
- A useful research agent needs durable memory.

Two memory sources

- external reports, filings, papers, market research
- prior experiments: what worked, failed, or remains unresolved

Sources: XAlpha: [Liu et al., 2026a].

Research example: XAlpha

- Report-to-Memory Absorption: reports become research memory
- Checks whether an idea can be implemented with available data
- Tri-alignment: hypothesis, code, and financial rationale agree

Key shift

The agent no longer treats every prompt as an isolated task. It accumulates research state across cycles.

Example: from report to research memory

External report says

"Momentum signals are more reliable when volatility is declining."

Agent checks

- Do we have price history? yes
- Do we have volatility proxies? yes
- Do we need unavailable order-book data? no

Memory entry

- Theme: regime-conditioned momentum
- Required data: OHLCV
- Testable hypothesis: momentum + falling volatility
- Next action: implement factor code and backtest

Why this matters

RAG retrieves text for the current answer. Research memory turns evidence into reusable future context.

The catch: agents can automate bad research too

Structural validity comes before performance

A spectacular backtest is not meaningful if the experiment used information, firms, or assumptions that would not exist in real deployment.

Five evaluation sins

- **Look-ahead:** future information leaks in
- **Survivorship:** failed firms disappear
- **Narrative:** fluent stories lack evidence
- **Objective:** confidence beats abstention
- **Cost:** frictions and latency ignored

Agentic failure example

- Agent runs thousands of experiments overnight.
- Evaluator accidentally retrieves future documents.
- Agent finds a “great” strategy.
- Result is not alpha; it is optimized leakage.

Key warning

An agent may be excellent at optimizing the evaluator. If the evaluator is invalid, the agent optimizes invalidity.

How agentic AI is changing finance

The major shift is not autonomous trading. It is controlled automation of financial research.

1. **Experimentation** becomes more autonomous.
2. **Search** expands from formulas to code and research trajectories.
3. **Memory** turns reports and prior failures into reusable research state.
4. **Verification** becomes more important, because agents scale both good and bad methodology.
5. **Production systems** prioritize attribution, reproducibility, integration, and accountability.

Takeaway

In finance, agentic capability must grow together with structural validity, controls, and human accountability.

Sources: Examples discussed: [Hogan et al., 2026, Liu et al., 2026b, Han et al., 2026, Liu et al., 2026a, Kong et al., 2026, Bloomberg, 2026].

Coming up: Hooman Sedghamiz

Block 4: Agents in Pharma & Life Sciences, and Deployment

- From financial decision support to scientific discovery and regulated workflows.
- Same architectural themes: bounded autonomy, verification, traceability, and governance.
- Different domain risks, but the same systems discipline.

Presented by **Hooman Sedghamiz** (Bayer).

References I

Anthropic (2024).

Building effective agents.
https://www.anthropic.com/research/building-effective-agents.

Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2023).

Self-RAG: Learning to retrieve, generate, and critique through self-reflection.
In *International Conference on Learning Representations*.

Bloomberg (2026).

Bloomberg unveils ASKB roadmap for clients to augment their investment process with agentic AI.
https://www.bloomberg.com/professional/insights/press-announcement/
bloomberg-unveils-askb-roadmap-for-clients-to-augment-their-investment-process-with-agentic-ai/.

Drouin, A., Gasse, M., Caccia, M., Laradji, I., Verme, M. D., Marty, T., Boisvert, L., Thakkar, M., Cappart, Q., Vázquez, D., et al. (2024).

WorkArena: How capable are web agents at solving common knowledge work tasks?
In *International Conference on Machine Learning*.

Fourney, A., Bansal, G., Mozannar, H., Tan, C., Salinas, E., Zhu, E., Niedtner, F., Proebsting, G., Bassman, G., Gerrits, J., et al. (2024).

Magentic-One: A generalist multi-agent system for solving complex tasks.
arXiv.org.

Gottweis, J., Weng, W.-H., Daryin, A., Tu, T., Sirkovic, P., Myaskovsky, A., Glowaty, G., Weissenberger, F., Orlandi, A., Popovici, D., et al. (2026).

Accelerating scientific discovery with co-scientist.
See also Gottweis et al., *Towards an AI co-scientist*, arXiv:2502.18864.

Gullí, A. (2025).

Model context protocol.
https://www.anthropic.com/news/model-context-protocol.

References II

Han, J., Zhang, S., Li, W., Yang, Z., Dong, Y., Hu, T., Yuan, J., Yu, X., Zhu, Y., Lou, F., et al. (2026).

QuantaAlpha: An evolutionary framework for LLM-driven alpha mining.

arXiv.org.

Hogan, B. R., Chen, X., Boyarsky, A., Kamei, T., Wilson, J. T., Rasul, K., Schneider, A., and Nevmyvaka, Y. (2026).

ALPHALAB: Autonomous multi-agent research across optimization domains with frontier LLMs.

arXiv.org.

Huang, K.-H., Prabhakar, A., Dhawan, S., Mao, Y., Wang, H., Savarese, S., Xiong, C., Laban, P., and Wu, C.-S. (2025).

CRMArena: Understanding the capacity of LLM agents to perform professional CRM tasks in realistic environments.

In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3830–3850. Association for Computational Linguistics.

Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., Wang, Y., Tang, R., and Chen, E. (2024).

Understanding the planning of LLM agents: A survey.

arXiv.org.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2024).

SWE-bench: Can language models resolve real-world GitHub issues?

In *International Conference on Learning Representations*.

Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., and Narayanan, A. (2024).

AI agents that matter.

Trans. Mach. Learn. Res.

Kong, Y., Lee, H., Hwang, Y., Lopez-Lira, A., Levy, B., Mehta, D., Wen, Q., Choi, C., Lee, Y., and Zohren, S. (2026).

Evaluating LLMs in finance requires explicit bias consideration.

arXiv.org.

References III

Liu, F., Fu, Y., Wang, Y., and Liu, Q. (2026a).

XALPHA: A memory-driven AI quant researcher for hypothesis-to-code alpha discovery.

arXiv.

Liu, F., Yi, H., Luo, S., Wang, Y., Yang, Y., Li, X., Hu, Z., Feng, J., and Liu, Q. (2026b).

Cognitive alpha mining via LLM-driven code-based evolution.

In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11715–11749. Association for Computational Linguistics.

Messing, B. (2002).

An Introduction to MultiAgent Systems.

John Wiley & Sons, 2 edition.

Mozannar, H., Bansal, G., Tan, C., Fourney, A., Dibia, V. C., Chen, J., Gerrits, J., Payne, T., Maldaner, M. K., Grunde-McLaughlin, M., et al. (2025).

Magentic-UI: Towards human-in-the-loop agentic systems.

arXiv.org.

Russell, S. J. and Norvig, P. (2020).

Artificial Intelligence: A Modern Approach.

Prentice Hall, 3 edition.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023).

Toolformer: Language models can teach themselves to use tools.

In *Advances in Neural Information Processing Systems 36*, volume 36, pages 68539–68551. Neural Information Processing Systems Foundation, Inc. (NeurIPS).

Shinn, N., Cassano, F., Labash, B., Gopinath, A., Narasimhan, K., and Yao, S. (2023).

Reflexion: Language agents with verbal reinforcement learning.

Neural Information Processing Systems, pages 8634–8652.

References IV

Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O'Sullivan, B., and Nguyen, H. D. (2025).
Multi-agent collaboration mechanisms: A survey of LLMs.
arXiv.org.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017).
Attention is all you need.
In *Neural Information Processing Systems*, volume 30. Shenzhen Medical Academy of Research and Translation.

Venkit, P. N., Laban, P., Zhou, Y., Huang, K.-H., Mao, Y., and Wu, C.-S. (2025).
DeepTRACE: Auditing deep research AI systems for tracking reliability across citations and evidence.
arXiv.org.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. (2024).
Voyager: An open-ended embodied agent with large language models.
Trans. Mach. Learn. Res.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E. H., and Zhou, D. (2022).
Self-consistency improves chain of thought reasoning in language models.
In *International Conference on Learning Representations*.

Wei, H., Zhang, Z., He, S., Xia, T., Pan, S., and Liu, F. (2025).
PlanGenLLMs: A modern survey of LLM planning capabilities.
In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 19497–19521. Association for Computational Linguistics.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E. H., Xia, F., Le, Q., and Zhou, D. (2022).
Chain-of-thought prompting elicits reasoning in large language models.
In *Neural Information Processing Systems*, volume 35, pages 24824–24837. Neural Information Processing Systems Foundation, Inc. (NeurIPS).

References V


Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al. (2023).
AutoGen: Enabling next-gen LLM applications via multi-agent conversation.
arXiv.


Yang, G. H., Venkit, P. N., Sedghamiz, H., Santus, E., Dibia, V., and Baldini, I. (2026).
Agents in the wild: Where research meets deployment.
KDD 2026 Tutorial.


Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. (2023).
Tree of thoughts: Deliberate problem solving with large language models.
In *Advances in Neural Information Processing Systems 36*, pages 11809–11822. Neural Information Processing Systems Foundation, Inc. (NeurIPS).


Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2022).
ReAct: Synergizing reasoning and acting in language models.
In *International Conference on Learning Representations*.


Yehudai, A., Eden, L., Li, A., Uziel, G., Zhao, Y., Bar-Haim, R., Cohan, A., and Shmueli-Scheuer, M. (2026).
Survey on evaluation of LLM-based agents.
Findings of the Association for Computational Linguistics: ACL 2026, pages 26690–26714.


Zhao, B., Foo, L. G., Hu, P., Theobalt, C., Rahmani, H., and Liu, J. (2025).
LLM-based agentic reasoning frameworks: A survey from methods to scenarios.
arXiv.org.


Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Bisk, Y., Fried, D., Alon, U., et al. (2024).
WebArena: A realistic web environment for building autonomous agents.
In *International Conference on Learning Representations*.

References VI

Zhu, K., Du, H., Hong, Z., et al. (2025).
Multiagentbench: Evaluating the collaboration and competition of llm agents.
arXiv preprint arXiv:2503.01935.